System Architecture Specification: Data Sync Core v0.0.6

Status: Architecture Candidate
Scope: Durable data ingestion, media metadata sync, case-record convergence, and identity/custody lifecycle
Out of Scope: Live video, real-time telemetry, device control, safety interlocks, OR command routing

---

0. Core Principle

The system does not synchronize mutable databases.

It transfers custody of immutable authored records through idempotent, signed, durable receipts.

```
Device / Vision unit owns authored facts.
Room Hub owns first durable custody.
Central owns canonical reconciliation and projections.
EHR/export systems consume approved projections.
MinIO/object storage owns media payload durability.
```

The system must work when the cloud is down, the room network is unreliable, and devices move between rooms.

The design must not depend on Couchbase, Sync Gateway, LiteFS, Kafka, RabbitMQ, or any vendor sync engine.

Those products may be replaced by:

```
SQLite
custom mTLS batch sync
PostgreSQL or another open-source central database
MinIO or another S3-compatible object store
simple background workers
```

---

1. What This Layer Does

This layer handles durable records only:

```
1. Case/device events
2. Vision capture events
3. Media metadata
4. Media object references
5. Case-session lifecycle facts
6. Completion/reopen/correction facts
7. Surgeon/procedure/preference down-sync metadata
```

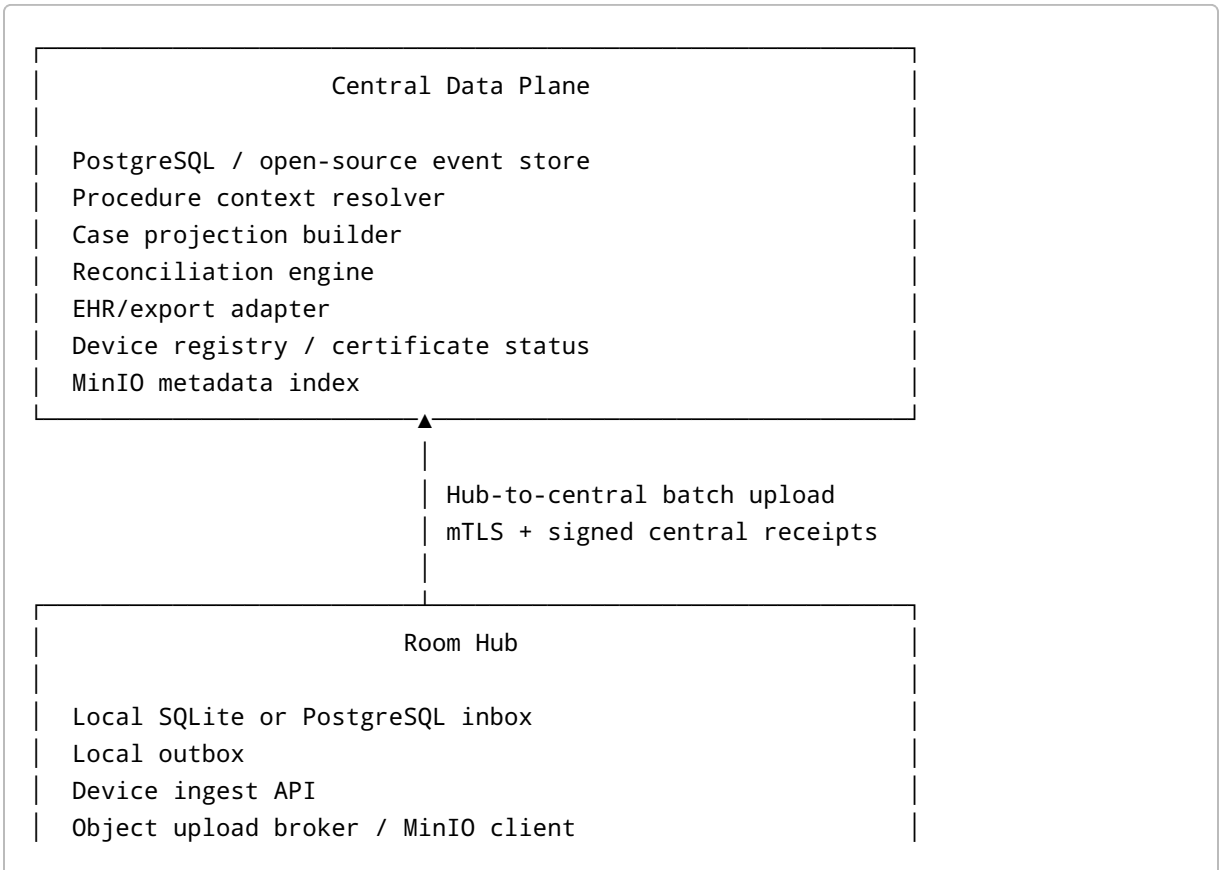
It does not carry:

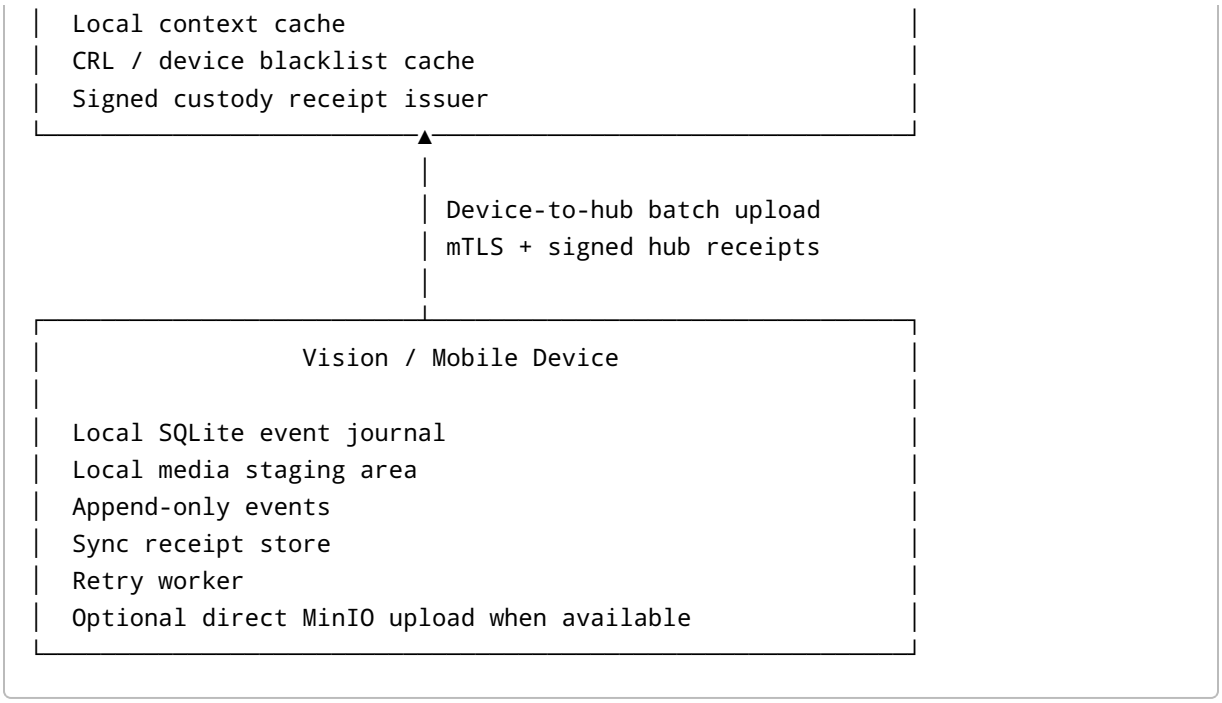
```
live arthroscopy video
real-time control commands
device actuation
safety interlocks
high-frequency telemetry streams
```

Live data and control stay outside this layer.

---

2. Architecture Shape





---

3. Golden Rule: No Shared Mutable Case Document

A clinical case must not be represented as one shared mutable document written by many offline-capable devices.

Bad model:

```
case::123
  status
  surgeon
  procedure
  images[]
  clips[]
  completed
  notes[]
```

This fails because multiple Vision units and central systems can mutate the same document independently.

Correct model:

```
A case is a namespace, not a mutable document.
```

A case is composed of separately owned records:

```
case_header::{case_id}                          central-owned
case_session::{case_id}::{device_id}            device-owned
case_event::{case_id}::{device_id}::{event_id} append-only
media_asset::{case_id}::{device_id}::{asset_id} create-once
media_upload::{asset_id}                        monotonic state machine
case_projection::{case_id}                      derived/rebuildable
```

Many devices may contribute to the same case.

They must not co-author the same mutable record.

---

4. Ownership Model

Record Type	Writer	Mutable?	Conflict Rule
`case_header`	Central	Yes	Central owns
`surgeon`	Central	Yes	Central owns
`procedure`	Central	Yes	Central owns
`surgeon_preference`	Central	Yes	Central owns
`case_session`	Owning Vision/ device	Limited	Owning device owns
`case_event`	Owning Vision/ device	No	Append-only
`media_asset`	Capturing Vision/ device	Create-once	Insert-or-compare
`media_upload`	Upload worker	Monotonic only	Forward state transitions only
`case_projection`	Projection worker	Rebuildable	Disposable
`central_correction_event`	Central/admin workflow	No	Append-only

There is no global “server wins” rule.

There is no global “device wins” rule.

The rule is:

```
The record owner wins only for records it is allowed to author.
Immutable facts are never overwritten.
Derived projections can be rebuilt.
```

---

5. Data Identity Boundary

Devices and room hubs must not persist patient identity.

Allowed below the central adapter edge:

```
case_id or procedure_context_id
room_id
device_id
session_id
event_id
asset_id
operator_session_ref
context_token_id
context_token_hash
device_time
device_seq
monotonic_ms
```

Forbidden below the central adapter edge:

```
patient name
MRN
DOB
admission ID
encounter ID
patient_ref
stable patient-resolving token
free-text patient notes
```

`case_id` / `procedure_context_id` must be opaque and non-derived.

The patient join happens only upstream:

```
procedure_context_id -> patient / encounter / EHR case
```

The device never sees this mapping.

6. Payload PHI Policy

The identity boundary applies to payloads too.

`payload_json` must not contain:

```
patient name
MRN
DOB
encounter ID
admission ID
patient notes
stable patient-resolving tokens
free-text PHI
```

Every event type must have an allowlisted schema.

Unknown fields are rejected or quarantined.

Free-text fields are forbidden at the device layer unless explicitly approved.

Table: `event_schema_registry`

```sql
CREATE TABLE event_schema_registry (
    event_type TEXT NOT NULL,
    schema_version INTEGER NOT NULL,

    payload_schema_hash TEXT NOT NULL,

    allows_free_text INTEGER NOT NULL DEFAULT 0,
    contains_phi INTEGER NOT NULL DEFAULT 0,

    status TEXT NOT NULL,
    -- active | deprecated | rejected

    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL,

    PRIMARY KEY(event_type, schema_version)
);
```

Device-originated event schemas must default to:

```
contains_phi = false
allows_free_text = false
```

---

7. Device Local Storage

Each Vision/mobile device uses local SQLite for durable authored records.

```
PRAGMA journal_mode = WAL;
PRAGMA synchronous = EXTRA;
PRAGMA foreign_keys = ON;
PRAGMA busy_timeout = 5000;
PRAGMA trusted_schema = OFF;
PRAGMA temp_store = MEMORY;
PRAGMA wal_autocheckpoint = 100;
```

`wal_autocheckpoint` is deployment-tuned.

The storage configuration must be validated under forced power-cut testing on the actual production OS, filesystem, flash controller, and firmware image.

---

8. Device Event Journal

Table: `device_events`

```
CREATE TABLE device_events (
    event_id TEXT PRIMARY KEY,
    device_id TEXT NOT NULL,
    device_seq INTEGER NOT NULL,

    boot_id TEXT NOT NULL,
    session_id TEXT,

    case_id TEXT,
    procedure_context_id TEXT,
    room_id TEXT,

    event_type TEXT NOT NULL,
    schema_version INTEGER NOT NULL DEFAULT 1,

    context_token_id TEXT,
    context_token_hash TEXT,
```

```
    context_mode TEXT NOT NULL DEFAULT 'normal',
    -- normal | emergency_override | unassigned | maintenance | test

    override_session_id TEXT,

    operator_session_ref TEXT,
    operator_context_token_hash TEXT,

    device_time TEXT NOT NULL,
    monotonic_ms INTEGER,

    payload_json TEXT NOT NULL,
    payload_hash TEXT NOT NULL,

    previous_event_hash TEXT,
    event_hash TEXT NOT NULL,
    event_signature TEXT,

    created_at_local TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,

    UNIQUE(device_id, device_seq)
);
```

Authoritative per-device ordering:

```
ORDER BY device_id, device_seq
```

Do not rely on ULID/UUIDv7 ordering for clinical chronology.

---

9. Receipt Storage

Receipts must store the full signed receipt payload, not only the signature.

Table: `sync_batch_receipts`

```
CREATE TABLE sync_batch_receipts (
    receipt_id TEXT PRIMARY KEY,

    destination_id TEXT NOT NULL,
    batch_id TEXT NOT NULL,
    receipt_type TEXT NOT NULL,
    -- hub_accepted | central_confirmed
```

```
    receipt_time TEXT NOT NULL,

    receipt_payload_json TEXT NOT NULL,
    receipt_signature TEXT NOT NULL,

    created_at_local TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Table: `sync_event_receipts`

```
CREATE TABLE sync_event_receipts (
    receipt_id TEXT NOT NULL,
    event_id TEXT NOT NULL,

    receipt_status TEXT NOT NULL,
    -- accepted | duplicate | rejected | conflict

    PRIMARY KEY(receipt_id, event_id),
    FOREIGN KEY(receipt_id) REFERENCES sync_batch_receipts(receipt_id),
    FOREIGN KEY(event_id) REFERENCES device_events(event_id)
);
```

---

10. Device Sync Cursor

```
CREATE TABLE sync_cursor (
    destination_id TEXT PRIMARY KEY,
    last_attempt_at TEXT,
    last_success_at TEXT,
    last_error TEXT
);
```

---

11. Device-to-Hub Sync Loop

The device never blocks clinical function to sync.

Sync is opportunistic and bounded.

```
1. Append facts locally.
2. Discover or already know the room hub endpoint.
```

```
3. Establish mTLS.
4. Validate or refresh signed context lease if available.
5. Select events without hub_accepted receipt.
6. Send a bounded batch.
7. Receive signed hub receipt.
8. Persist full receipt.
9. Retry later if anything fails.
```

Selection query:

```sql
SELECT e.*
FROM device_events e
LEFT JOIN sync_event_receipts er
  ON er.event_id = e.event_id
LEFT JOIN sync_batch_receipts br
  ON br.receipt_id = er.receipt_id
 AND br.destination_id = :hub_id
 AND br.receipt_type = 'hub_accepted'
WHERE br.receipt_id IS NULL
ORDER BY e.device_seq
LIMIT :batch_limit;
```

---

12. Batch Upload Protocol

Endpoint:

```
POST /v1/events/batch
Authorization: mTLS device certificate
Content-Type: application/json
Idempotency-Key: batch_id
```

Payload:

```json
{
  "batch_id": "batch-01JABC",
  "device_id": "vision-042",
  "events": []
}
```

The transport is custom and intentionally simple.

No database replication protocol is required.

---

13. Insert-or-Compare Rule

The hub and central must not blindly ignore duplicates.

They use insert-or-compare.

A safe duplicate requires all of these to match:

```
event_id
device_id
device_seq
event_hash
```

Rules:

```
New event:
  insert and accept

Same event_id + same device_id + same device_seq + same event_hash:
  classify as duplicate and return success

Same event_id but different device_id/device_seq:
  conflict

Same device_id/device_seq but different event_id:
  conflict

Same event_id/device_seq but different event_hash:
  conflict

Same hash but different event_id:
  suspicious; do not automatically dedupe
```

Conflicts are not merge conflicts.

They are integrity incidents.

---

14. Hub Receipt

The hub returns a signed receipt only after events are durably written or verified as exact duplicates.

```
{
  "receipt_id": "rcpt-01JABC",
  "hub_id": "hub-or-02",
  "device_id": "vision-042",
  "batch_id": "batch-01JABC",
  "received_at": "2026-07-04T13:41:15.033Z",
  "accepted_event_ids": [
    "evt-01"
  ],
  "duplicate_event_ids": [],
  "conflict_event_ids": [],
  "rejected": [],
  "signature": "hub-signature"
}
```

The signature covers:

```
receipt_id
hub_id
device_id
batch_id
received_at
accepted_event_ids
duplicate_event_ids
conflict_event_ids
rejected
```

---

15. Room Hub Storage

The Room Hub is the first durable custody point.

It can use SQLite for small/local deployments or PostgreSQL for larger room/site deployments.

Table: `hub_event_inbox`

```
CREATE TABLE hub_event_inbox (
    event_id TEXT PRIMARY KEY,
    device_id TEXT NOT NULL,
    device_seq INTEGER NOT NULL,

    case_id TEXT,
    procedure_context_id TEXT,
```

```
    source_room_id TEXT,

    event_type TEXT NOT NULL,
    schema_version INTEGER NOT NULL,

    context_token_id TEXT,
    context_token_hash TEXT,
    context_mode TEXT NOT NULL,

    override_session_id TEXT,

    operator_session_ref TEXT,
    operator_context_token_hash TEXT,

    received_by_hub_id TEXT NOT NULL,
    received_at_hub TEXT NOT NULL,

    device_time TEXT NOT NULL,
    monotonic_ms INTEGER,

    payload_json TEXT NOT NULL,
    payload_hash TEXT NOT NULL,

    previous_event_hash TEXT,
    event_hash TEXT NOT NULL,
    event_signature TEXT,

    central_status TEXT NOT NULL DEFAULT 'pending',

    UNIQUE(device_id, device_seq)
);
```

Table: `hub_outbox`

```
CREATE TABLE hub_outbox (
    outbox_id TEXT PRIMARY KEY,
    event_id TEXT NOT NULL,

    destination_id TEXT NOT NULL DEFAULT 'central',

    status TEXT NOT NULL DEFAULT 'pending',
    -- pending | in_flight | confirmed | failed | quarantined

    attempt_count INTEGER NOT NULL DEFAULT 0,
    last_attempt_at TEXT,
    next_attempt_at TEXT,
```

```
    last_error TEXT,

    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY(event_id) REFERENCES hub_event_inbox(event_id)
);
```

Table: `hub_integrity_incidents`

```
CREATE TABLE hub_integrity_incidents (
    incident_id TEXT PRIMARY KEY,

    event_id TEXT,
    device_id TEXT NOT NULL,
    device_seq INTEGER,

    incident_type TEXT NOT NULL,
    -- duplicate_hash_mismatch
    -- invalid_signature
    -- sequence_gap_detected
    -- invalid_context_token
    -- expired_context_token
    -- malformed_payload
    -- blacklisted_device

    details_json TEXT NOT NULL,

    detected_at_hub TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'open'
);
```

---

16. Hub-to-Central Sync

The hub uses the same custody pattern to upload to central.

```
hub_event_inbox
      ↓
hub_outbox
      ↓
POST /v1/hub-events/batch
      ↓
central_event_store
```

```
      ↓
signed central receipt
```

The hub marks an event as `central_status = confirmed` only after a valid central receipt is stored.

The hub must batch upload, not stream.

---

17. Central Event Store

Central is the canonical store for all accepted facts.

PostgreSQL is the preferred open-source implementation.

Table: `central_events`

```
CREATE TABLE central_events (
    event_id TEXT PRIMARY KEY,
    device_id TEXT NOT NULL,
    device_seq INTEGER NOT NULL,

    case_id TEXT,
    procedure_context_id TEXT,
    source_room_id TEXT,

    event_type TEXT NOT NULL,
    schema_version INTEGER NOT NULL,

    context_token_id TEXT,
    context_token_hash TEXT,
    context_mode TEXT NOT NULL,

    override_session_id TEXT,

    operator_session_ref TEXT,
    operator_context_token_hash TEXT,

    received_by_hub_id TEXT NOT NULL,
    received_at_hub TEXT NOT NULL,
    received_at_central TEXT NOT NULL,

    device_time TEXT NOT NULL,
    monotonic_ms INTEGER,

    payload_json TEXT NOT NULL,
```

```
    payload_hash TEXT NOT NULL,

    previous_event_hash TEXT,
    event_hash TEXT NOT NULL,
    event_signature TEXT,

    UNIQUE(device_id, device_seq)
);
```

---

18. Case Namespace Model

A case is a namespace.

It contains separately owned records.

```
case_header
case_session
case_event
media_asset
media_upload
case_projection
central_correction_event
```

No device writes a shared mutable case document.

If a legacy UI requires a single case object, it reads `case_projection`.

`case_projection` is disposable and rebuildable.

---

19. Case Header

Central owns case header records.

Table: `case_headers`

```
CREATE TABLE case_headers (
    case_id TEXT PRIMARY KEY,
    procedure_context_id TEXT UNIQUE,

    site_id TEXT,
    room_id TEXT,
```

```
    surgeon_ref TEXT,
    procedure_ref TEXT,

    scheduled_start TEXT,
    scheduled_end TEXT,

    status TEXT NOT NULL,
    -- scheduled | active | cancelled | closed

    revision INTEGER NOT NULL DEFAULT 1,

    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL
);
```

This table may map to EHR data upstream, but devices must only receive opaque identifiers.

---

20. Case Sessions

Each Vision/device writes its own case session.

Table: `case_sessions`

```
CREATE TABLE case_sessions (
    session_id TEXT PRIMARY KEY,

    case_id TEXT NOT NULL,
    procedure_context_id TEXT,
    device_id TEXT NOT NULL,
    room_id TEXT,

    started_at_device TEXT,
    ended_at_device TEXT,

    session_state TEXT NOT NULL,
    -- active | locally_completed | abandoned | reconciled

    sequence_high_watermark INTEGER,

    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL,
```

```
    UNIQUE(case_id, device_id, session_id)
);
```

A device may update only its own session.

A session completion is not the same thing as legal case completion.

---

21. Case Events

Clinical milestones are events, not mutable fields.

Examples:

```
case_started
case_completed
case_reopened
device_session_started
device_session_completed
image_captured
clip_recorded
media_upload_started
media_upload_completed
media_upload_failed
completion_corrected
operator_association_corrected
emergency_events_reconciled
```

Completion must be represented as:

```
case_completed event
```

not:

```
case.status = completed
```

If multiple devices report completion, central reconciliation keeps all facts and computes the projection.

No completion event is erased by a later server update.

---

22. Media Asset Model

Media payloads are stored in MinIO or another S3-compatible object store.

The DB stores only metadata and object references.

Table: `media_assets`

```
CREATE TABLE media_assets (
    asset_id TEXT PRIMARY KEY,

    case_id TEXT NOT NULL,
    procedure_context_id TEXT,

    session_id TEXT,
    device_id TEXT NOT NULL,
    device_seq INTEGER,

    media_type TEXT NOT NULL,
    -- still | clip | segment | thumbnail

    captured_at_device TEXT NOT NULL,
    received_at_hub TEXT,

    object_bucket TEXT NOT NULL,
    object_key TEXT NOT NULL,

    object_sha256 TEXT NOT NULL,
    object_size_bytes INTEGER,

    upload_state TEXT NOT NULL,
    -- local_only | hub_staged | object_uploaded | central_indexed | failed

    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL,

    UNIQUE(device_id, device_seq),
    UNIQUE(object_bucket, object_key)
);
```

Media objects must be encrypted at rest.

Access must use short-lived signed URLs or equivalent access tokens.

Large video/image blobs must not be stored in the event database.

23. Media Upload State Machine

Media upload state is monotonic.

Allowed forward transitions:

```
local_only
  -> hub_staged
  -> object_uploaded
  -> central_indexed
```

Failure may be recorded as:

```
failed
```

but the original capture event and object checksum must not be overwritten.

If upload is retried and the object hash matches, it is a duplicate success.

If the object key matches but the hash differs, it is an integrity incident.

---

24. Case Projection

The visible case is derived from source records.

Table: `case_projections`

```sql
CREATE TABLE case_projections (
    case_id TEXT PRIMARY KEY,

    computed_status TEXT NOT NULL,
    -- scheduled | active | completed | reopened | needs_review

    completed_at TEXT,
    completed_source_event_id TEXT,

    media_count INTEGER NOT NULL DEFAULT 0,
    still_count INTEGER NOT NULL DEFAULT 0,
    clip_count INTEGER NOT NULL DEFAULT 0,

    last_device_event_at TEXT,
```

```
    last_hub_receive_at TEXT,
    last_central_update_at TEXT,

    projection_version INTEGER NOT NULL,
    projection_payload_json TEXT NOT NULL,

    rebuilt_at TEXT NOT NULL
);
```

`case_projection` may be deleted and rebuilt.

It is not the source of truth.

---

25. Completion Rules

Default rule:

```
If at least one valid case_completed event exists
and no later authorized case_reopened or completion_corrected event overrides
it,
the projected case status is completed.
```

If multiple valid completion events exist:

```
Keep all events.
Use policy to pick projected completion:
  earliest valid completion
  primary device completion
  authorized room/session completion
  or manual review
```

Late media after completion may still be accepted if:

```
captured_at_device is within the case/session window
or the media belongs to a valid bound session
or an authorized reconciliation approves it
```

A server-side case metadata update must never revert a valid completion event.

---

26. Central Corrections

Corrections are append-only.

They never mutate source facts.

Table: `central_correction_events`

```
CREATE TABLE central_correction_events (
    correction_event_id TEXT PRIMARY KEY,

    correction_type TEXT NOT NULL,
    -- emergency_events_reconciled
    -- context_association_corrected
    -- operator_association_corrected
    -- duplicate_event_reviewed
    -- completion_corrected
    -- media_reassociated

    target_event_id TEXT,
    target_override_session_id TEXT,
    target_device_id TEXT,
    target_asset_id TEXT,
    target_case_id TEXT,

    correction_payload_json TEXT NOT NULL,

    created_by TEXT NOT NULL,
    created_at TEXT NOT NULL,
    reason TEXT NOT NULL
);
```

Corrections are applied by projections and read models.

Original facts remain unchanged.

---

27. Context Lease

Context binding is an explicit signed handshake.

The device may discover a hub endpoint, but no PHI or case metadata is exchanged over unauthenticated discovery.

Context Token

```
{
  "token_id": "tok-99312",
  "issuer": "hub-or-02",
  "issuer_key_id": "hub-or-02-key-7",
  "device_id": "vision-042",
  "room_id": "OR-02",
  "case_id": "case-123",
  "procedure_context_id": "ctx-proc-01JABC",
  "issued_at": "2026-07-04T13:30:00Z",
  "valid_from": "2026-07-04T13:30:00Z",
  "valid_until": "2026-07-04T18:30:00Z",
  "binding_method": "staff_confirmed",
  "confirmed_by": "opaque-operator-session",
  "signature": "hub-signature"
}
```

The token must not contain patient identity.

---

28. Emergency Override

Emergency override is explicit, not inferred from null fields.

When device use begins without a valid context:

```
{
  "event_type": "emergency_override_started",
  "context_mode": "emergency_override",
  "override_session_id": "ovr-01JABC",
  "case_id": null,
  "procedure_context_id": null
}
```

Events in this mode carry:

```
context_mode = emergency_override
override_session_id = ...
case_id = null
procedure_context_id = null
```

Central later appends:

```
emergency_events_reconciled
```

to associate the override session with a real case.

No original event is modified.

---

29. Operator Identity Boundary

Devices must not store direct staff identity.

Do not store:

```
clinician name
badge number
employee ID
email
NPI
persistent staff ID
```

Store only:

```
operator_session_ref
operator_context_token_hash
```

These are opaque, scoped, short-lived, and resolvable only upstream.

---

30. Cryptographic Signing

Events are canonicalized before hashing/signing.

Canonicalization rules:

```
UTF-8
lexicographic JSON key ordering
no insignificant whitespace
stable integer representation
stable decimal representation
```

```
ISO8601 UTC timestamps
payload_json hashed separately into payload_hash
```

Signed envelope:

```
sign(
  device_id,
  device_seq,
  event_id,
  boot_id,
  session_id,
  case_id,
  procedure_context_id,
  room_id,
  event_type,
  schema_version,
  context_token_id,
  context_token_hash,
  context_mode,
  override_session_id,
  operator_session_ref,
  operator_context_token_hash,
  device_time,
  monotonic_ms,
  payload_hash,
  previous_event_hash,
  event_hash
)
```

Hash chaining provides tamper evidence and gap detection.

Hardware-backed signatures strengthen authorship verification.

---

31. Identity and Provisioning

Use two separate identity roles:

```
Factory attestation identity:
  proves the hardware is genuine.

Hospital operational identity:
  proves the device is enrolled and authorized at this hospital/site.
```

Do not make the hospital operational root a child of the manufacturer root.

The hub validates both:

```
genuine hardware
currently enrolled operational identity
not revoked
not decommissioned
```

---

32. Field Enrollment

For field enrollment:

```
1. Admin issues short-lived Technician Onboarding Token.
2. Token is single-use.
3. Token is bound to technician identity.
4. Token is bound to device_id or serial.
5. Technician uses physical maintenance port.
6. Device generates hardware-backed CSR.
7. Enrollment service signs operational certificate.
8. Technician terminal never holds root or intermediate CA keys.
9. Enrollment is logged centrally and, when possible, on device.
```

---

33. Revocation and CRL Cache

Room hubs must support offline identity validation.

Each hub tracks:

```
crl_version
crl_generated_at
crl_expires_at
last_successful_crl_sync_at
max_stale_duration
```

Policy:

```
Fresh CRL:
  accept valid non-revoked device
```

```
Stale but within grace window:
  accept valid device and mark identity_validation_status = stale_crl

Beyond max stale duration:
  reject, quarantine, or emergency-only accept according to hospital policy
```

---

34. Graded Quarantine

Not all identity failures are equal.

Condition	Behavior
Revoked certificate	Reject connection
Decommissioned device	Reject connection
Malformed certificate	Reject connection
Unknown device	Reject or quarantine
Recently expired operational cert	Reject or quarantine by policy
Stale CRL	Degraded acceptance or quarantine
Valid cert, invalid event signature	Reject affected events and open incident
Valid cert, expired context token	Accept as unassigned or reject context-bound fields
Valid cert, malformed payload	Reject affected events and open incident

---

35. Preference Down-Sync

Surgeon/procedure/preferences are central-authored reference records.

They sync down to hubs/devices as read-only cached data.

Rules:

```
Central owns preferences.
Edge reads preferences.
Device must not directly change behavior from a preference document unless an
authorized planner/workflow applies it.
```

```
Offline devices may use last-known valid preferences.
Preference updates do not overwrite case events or media facts.
```

Preference records are separate from case events.

---

36. Offline Behavior

The system must work during network loss.

Device offline

```
Device continues writing local events.
Device continues staging media locally.
No central system is required for local authoring.
```

Hub offline from central

```
Hub accepts device batches.
Hub returns hub_accepted receipts.
Hub queues central upload.
```

Device dies after sending but before saving receipt

```
Device retries.
Hub detects exact duplicate.
Hub returns duplicate success.
```

Multiple Vision devices offline

```
Each writes its own session/events/assets.
No shared document conflict occurs.
Central reconciles after sync.
```

---

37. Cleanup and Retention

Device cleanup default:

```
hub_accepted receipt exists
and local retention window has passed
```

Optional stricter policy:

```
central_confirmed receipt exists
and local retention window has passed
```

Hub cleanup:

```
central_confirmed receipt exists
and hub retention window has passed
```

Central retention follows institutional policy.

Media object retention follows MinIO/object-store lifecycle policy.

---

38. Minimal Open-Source Deployment

Smallest deployment:

```
Device / Vision:
  SQLite event journal
  local media staging
  mTLS client
  sync worker

Room Hub:
  SQLite inbox/outbox
  HTTPS/mTLS ingest API
  signed receipt issuer
  MinIO upload worker
  central sync worker
  CRL cache

Central:
  PostgreSQL
  reconciliation workers
  projection builder
  EHR/export adapter
```

```
  device registry
  MinIO
```

No Couchbase.

No Sync Gateway.

No Kafka required.

No distributed database required.

No global shared mutable case document.

---

39. Bottom Line

The data-sync design is:

```
central-owned reference data down
device-owned immutable facts up
media payloads to MinIO
metadata through custom custody sync
case state from projections
completion as event
corrections as events
no shared mutable case document
```

The most important rule:

```
Many devices may contribute to the same case.
They must never co-author the same mutable record.
```